MODULE 2 · STATISTICS & PROBABILITY · TOPICS 2.1 – 2.3

Descriptive Statistics

Day 23 — centre, spread, and shape



Boston University

90 min	30 min	1
Concepts + live code	Practice notebook	Question you have asked 3 times

Prepared & presented by **Srinivasa Sai Chava**

MODULE 1 → MODULE 2

You have been computing these for nine sessions

What you already do

```
df["salary"].mean()
df["salary"].std()
df.describe()
df.groupby("city")["v"].median()
```

All of it correct. None of it explained.

The question you have met three times

Day 14: “NumPy std uses ddof=0.”

Day 15: “pandas std uses ddof=1.”

Day 19: “state ddof explicitly.”

Each time I said “we will come back to why.”
Today is that session.

What changes in Module 2

In Module 1 your data WAS the thing you cared about: this spreadsheet, these 423 orders. From today, your data is a SAMPLE and you are trying to say something about a larger world you cannot see — every customer, not just the ones who bought in 2025. That one shift is the whole of statistics, and it is why the formulas change. Everything else in this module follows from it.

SESSION PLAN

How today's two hours are structured

1 **Centre** — 25 min

Mean, median, mode — and when each lies

2 **Spread** — 35 min

Range, IQR, variance, SD — and ddof

3 **Shape** — 25 min

Skewness and kurtosis

4 **Mini build** — 5 min

A describe() that tells the truth

5 **Practice notebook** — 30 min

You code, I walk around

Section 2 gets the most time

Because that is where ddof lives, and I am going to prove it by simulation rather than algebra.

LEARNING OUTCOMES

By the end of today you will be able to

1

Choose

Pick mean, median or mode and defend the choice

2

Quantify

Describe spread with the right measure for the data

3

Explain

Say what n-1 is for, and why pandas and NumPy disagree

4

Read

Tell two distributions apart when the mean and SD match

Three numbers describe any distribution

Where is it (centre), how spread out is it (spread), and what shape is it (skew and tails). Day 20's Anscombe quartet showed you that the first two are not enough. Today you get the third, and a rule for when each summary is honest.

1

Centre

Mean, median, mode · 25 minutes

- The three measures
- When the mean lies
- Choosing

Slides by Srinivasa Sai Chava

Three answers to “what is typical?”

Measure	Is	pandas	Best when
Mean	the balance point	`s.mean()`	roughly symmetric, no extremes
Median	the middle value	`s.median()`	skewed, or outliers present
Mode	the most common value	`s.mode()`	categories, or repeated values

The mean is the only one that uses EVERY value

That is its strength and its weakness in one sentence. Every observation pulls on it, so it uses all your information — and so one absurd value drags it anywhere you like.

The median only cares about ORDER, so an extreme value counts for exactly one row, however extreme it is.

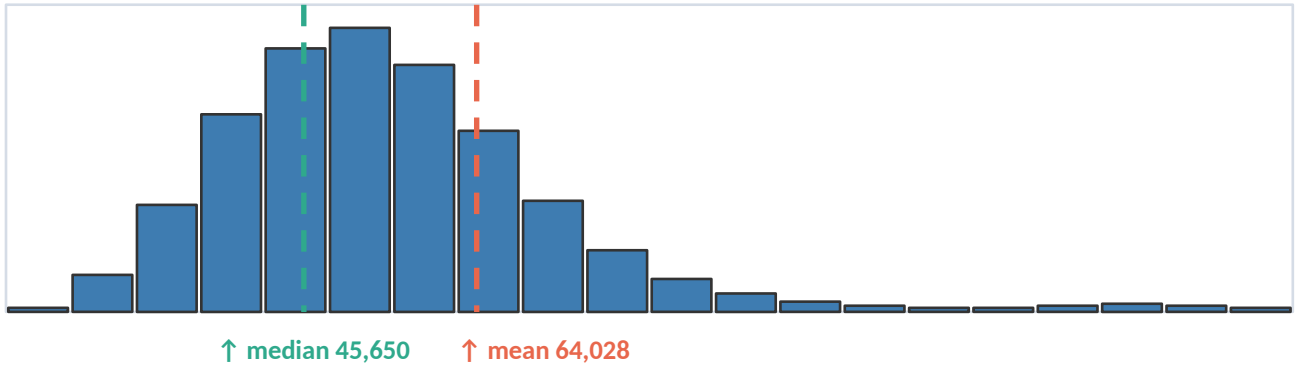
⚠️ mode() returns a SERIES

```
s.mean()      ->  a number
s.median()    ->  a number
s.mode()      ->  a Series!
s.mode()[0]   ->  a number
```

Because data can have two equally common values, pandas has to return all of them. Taking [0] silently hides a tie — check len(s.mode()) first.

When the mean describes nobody

1,000 salaries: 950 ordinary, 50 executives



The 50 executives at the far right pull the mean 18,000 above the median.

⚠️ The number that matters

94.1%

of these people earn LESS than the average salary.

"The average salary here is 64,000" is arithmetically true and completely misleading.

The rule

When the mean and the median differ substantially, report BOTH — the gap between them IS the finding. This is Day 21's "average salary" case again, and it is why every serious income statistic you will ever read is a median.

2

Spread

And the ddof question, finally · 35 minutes

- Range and IQR
- Variance and SD
- Why n - 1
- Population vs sample

Four ways to say “how spread out?”

Measure	Definition	Robust?	Units
Range	max - min	No — uses only the extremes	same as data
IQR	Q3 - Q1, the middle 50%	Yes — ignores the tails	same as data
Variance	mean squared distance from the mean	No	data units SQUARED
Std deviation	the square root of the variance	No	same as data

Why bother squaring at all?

The obvious measure — the average distance from the mean — is always exactly ZERO, because the positives and negatives cancel. That is what “balance point” means.

Squaring removes the signs. Then the square root puts you back into the original units, which is why we quote the SD and not the variance.

Reading an SD

```
mean 100, sd 15
  about 68% fall in  85 - 115
  about 95% fall in  70 - 130
```

Verified by simulation: 68.1% and 95.5%. This is the empirical rule, and it only holds for roughly bell-shaped data — Day 24's topic.

Why divide by n − 1? Here is the proof.

The experiment: 500,000 samples of 5, from a population we KNOW

```
sim = np.random.default_rng(2026)
POP = sim.normal(100, 15, 1_000_000)
true variance of POP                  225.06

samples = sim.choice(POP, size=(500_000, 5))
samples.var(axis=1, ddof=0).mean()    180.27   <- 20% low
samples.var(axis=1, ddof=1).mean()    225.34   <- on target
```

Dividing by n UNDERSHOOTS the truth, every time, on average.
Dividing by n − 1 lands on it.

And the shortfall is not arbitrary

It tracks (n−1)/n at EVERY sample size:

n	observed	(n−1)/n
3	0.666	0.667
5	0.800	0.800
10	0.899	0.900
50	0.980	0.980
200	0.995	0.995

The bias IS the factor you correct by — and it fades as n grows, which is why nobody notices it on large samples.

The intuition in one sentence

You measured each value's distance from the SAMPLE mean — but that mean was calculated from those same values, so it sits closer to them than the true population mean does. Your distances are therefore too small, always. Dividing by the smaller number n − 1 inflates the answer by exactly the right amount.

Population or sample? The question behind the flag

ddof=0 — population

Use when your data IS everything you care about:

- the marks of all 60 students in this class
- every order the shop took in 2025
- all 50 states

You are describing, not inferring. Nothing is unseen.

ddof=1 — sample

Use when your data stands in for something bigger:

- 200 customers, to describe all customers
- a month of sensor readings, to describe the sensor
- survey respondents, to describe the population

This is almost always the case, which is why pandas made it the default.

⚠️ And now the Day 14 warning makes sense

```
np.std(x)           ddof=0   population
pd.Series(x).std()  ddof=1   sample
#  same data, different answers
```

The two libraries chose different defaults because they were built for different jobs: NumPy for numerical arrays of whatever you have, pandas for sampled tabular data. Neither is wrong. State ddof explicitly whenever the number will be reported to anyone.

3

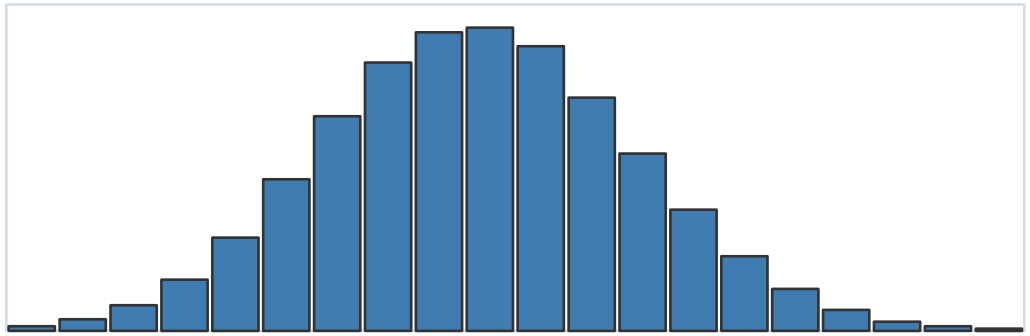
Shape

Skewness and kurtosis · 25 minutes

- Why centre and spread are not enough
- Skewness
- Kurtosis

Same mean. Similar SD. Completely different data.

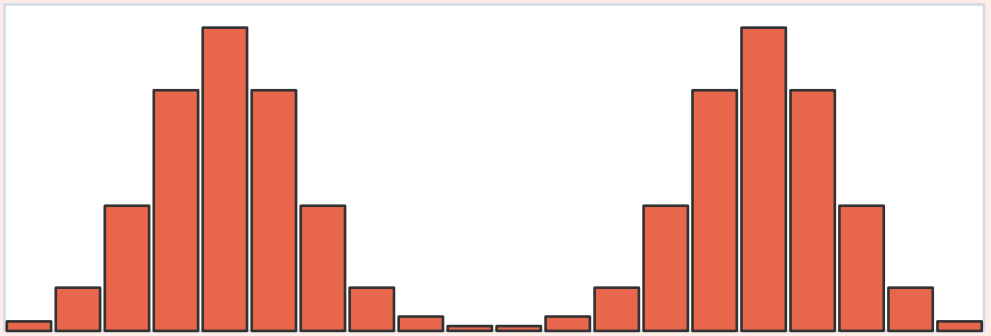
A — one hump



`mean 49.9  sd 10.0  kurtosis  0.00`

Most people are near 50.

B — two humps



`mean 50.0  sd 10.8  kurtosis −1.49`

Almost NOBODY is near 50. Two groups.

The mean of B describes a person who does not exist

Report “mean 50, SD 10” for either and the reader pictures A. Day 20's Anscombe made this point for relationships; this is the same point for distributions. Centre and spread are two numbers, and two numbers cannot describe a shape — which is why you plot, and why skewness and kurtosis exist.

Two numbers for shape

Skewness — which side is the tail?

Value	Means	Example
> 0	long tail to the RIGHT	income, delays
≈ 0	symmetric	heights
< 0	long tail to the LEFT	exam marks with a ceiling

Rough guide: |skew| above 1 is strongly skewed — use the median.

Kurtosis — how heavy are the tails?

Value	Means
> 0	heavy tails — more extremes than normal
≈ 0	normal-like tails
< 0	light tails — flat, or two humps

pandas reports EXCESS kurtosis, so a normal curve scores 0 rather than 3. Other tools differ — check before comparing.

The practical reading

```
s.skew()     4.28    strongly right-skewed
s.kurt()    24.90    very heavy tails
#  -> report the median, expect outliers
```

You rarely quote these numbers to a client. You use them to decide WHICH summary to quote: a skew of 4.28 tells you the mean will mislead before you have plotted anything, and a high kurtosis warns you that extreme values are coming.

Choosing the honest summary

If the data is…	Centre	Spread	Because
Roughly symmetric	mean	std	every value contributes, nothing distorts
Skewed	median	IQR	the mean chases the tail
Full of outliers	median	IQR	quartiles ignore the extremes
Bimodal (two humps)	neither — split it	—	one summary describes no one
Categorical	mode	counts	there is no arithmetic to do

And always report n

“The average is 55,000” is not a statistic. “The median is 45,650 across 1,000 staff, with a mean of 64,028 pulled up by 50 executives” is. Day 19 taught you to report how many values a summary used; today adds which summary, and why. Those two habits together are most of what separates a statistician from someone who can call .mean().

RECAP

Debug corner — eight statistics mistakes

You did	What goes wrong	Fix
`s.mean()  on skewed data`	describes almost nobody	`s.median()  — report both`
`s.mode()[0]`	silently hides a tie	`check len(s.mode())`
`np.std  vs  Series.std`	different answers, same data	state ddof explicitly
`ddof=0  on a sample`	understates the spread	`ddof=1`
quoting the variance	units are squared — meaningless	quote the std deviation
mean and SD on two humps	a summary describing no one	plot it, then split
comparing kurtosis across tools	pandas reports EXCESS kurtosis	check the convention
a summary with no n	unverifiable, and often wrong	always report the count

Rows 3 and 4 are the ones you have already met three times. Now you can say why they matter.

Putting it together — a describe() that tells the truth

```python
import pandas as pd

def honest_summary(s, name="value"):
    """Centre, spread and shape - and a recommendation."""
    s = s.dropna()
    skew = s.skew()
    out = {
        "n":        len(s),
        "missing":  s.isna().sum(),
        "mean":     round(s.mean(), 2),
        "median":   round(s.median(), 2),
        "std":      round(s.std(), 2),          # ddof=1
        "std_pop":  round(s.std(ddof=0), 2),    # ddof=0
        "IQR":      round(s.quantile(.75) - s.quantile(.25), 2),
        "skew":     round(skew, 2),
        "kurtosis": round(s.kurt(), 2),
    }

    #  the part describe() will not do for you
    gap = abs(out["mean"] - out["median"]) / out["std"]
    if abs(skew) > 1 or gap > 0.3:
        out["report"] = "MEDIAN and IQR - the data is skewed"
    else:
        out["report"] = "mean and std - roughly symmetric"
    return pd.Series(out)
```

- **dropna first** — every statistic needs a clean n
- **n AND missing** — Day 19's rule, kept
- **mean beside median** — the gap is the finding
- **both ddof values** — so the choice is visible
- **IQR alongside std** — robust and non-robust
- **skew and kurtosis** — shape, not just centre
- **a recommendation** — the line describe() never gives

describe() gives you numbers. This gives you a decision — which is what you were actually after.

OVER TO YOU

Practice session — 30 minutes

A **Predict the output** — 8 questions

Mean vs median, ddof, mode, skew signs

B **Spot & fix the bug** — 4 questions

Wrong ddof, mode[0], variance units

C **Write the code** — 5 tasks

Summarise, compare, detect skew, choose

Your notebook

`Day23_Practice_Questions.ipynb`

- Report n with every summary
- Mean AND median when skewed
- Raise a hand — I am walking around
- Solutions shared after class

For every summary you compute today, write the one sentence you would say to a non-statistician.

TODAY IN TWELVE LINES

Recap

1. Describe any distribution with centre, spread and shape.
2. The mean uses every value — its strength and its weakness.
3. The median only cares about order, so outliers count once.
4. Mean far from median = skew. Report BOTH; the gap is the finding.
5. In the salary example, 94% of people earn below the average.
6. mode() returns a Series, because ties are possible.
7. Variance is in squared units; quote the standard deviation.
8. ⚠️ Dividing by n understates the truth by exactly (n−1)/n.
9. So ddof=1 for a SAMPLE, ddof=0 for a whole POPULATION.
10. NumPy defaults to 0, pandas to 1. Neither is wrong. State it.
11. Skew says which tail; kurtosis says how heavy. Both guide the choice.
12. Two humps can share a mean and SD with one. Always plot.

Homework

Reproduce the n−1 simulation yourself and try n = 3, 10 and 50 · Find a real dataset where the mean and median disagree, and explain why · Run honest_summary on three columns of your Day 21 project data

Next class — Topic 2.4

The normal distribution: the bell curve, the 68-95-99.7 rule, and z-scores — where today's standard deviation becomes a ruler you can measure anything with.